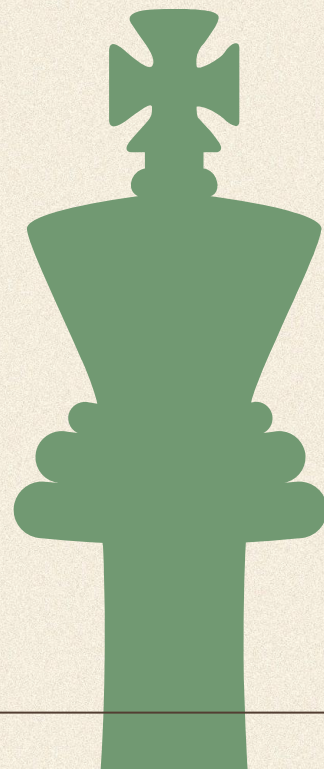
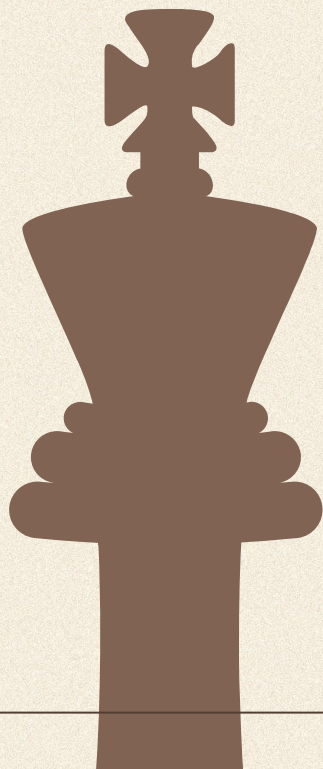


Joseph Hoane Chess Robot

Demo 2

Jordan Rios, Brayden Hayworth, Connor
Tynan, Evan Litzer

IOWA STATE
UNIVERSITY

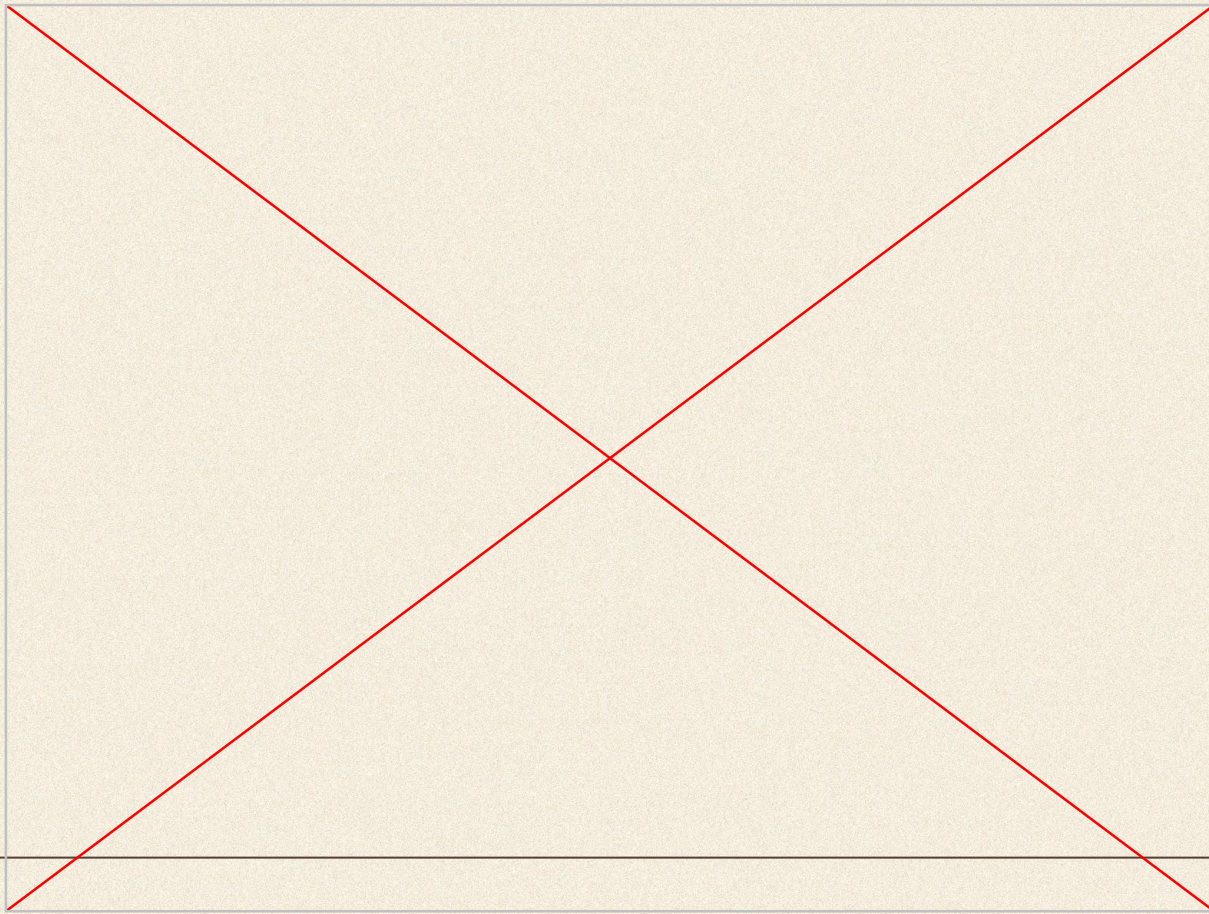


Problem

- Autonomous chess playing robot requires bridging three domains simultaneously.
 - Computer vision module → reading the board
 - Game intelligence engine → computing moves and managing game flow.
 - Robotic control → physically moving pieces on chess board.
 - Every part of the system depends on another
- **No accessible, integrated system for robotic arm to autonomously play physical chess against a human.**
- Need real-time, accurate, and consistent coordination of system.



Demo Video

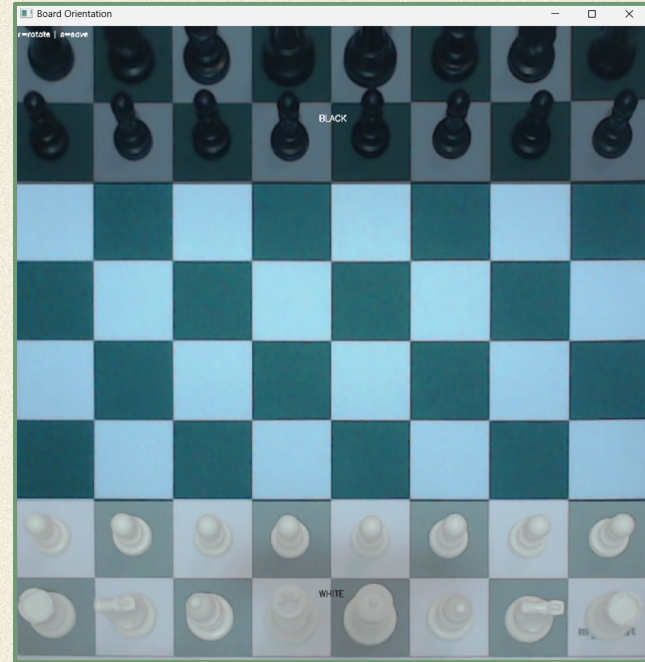
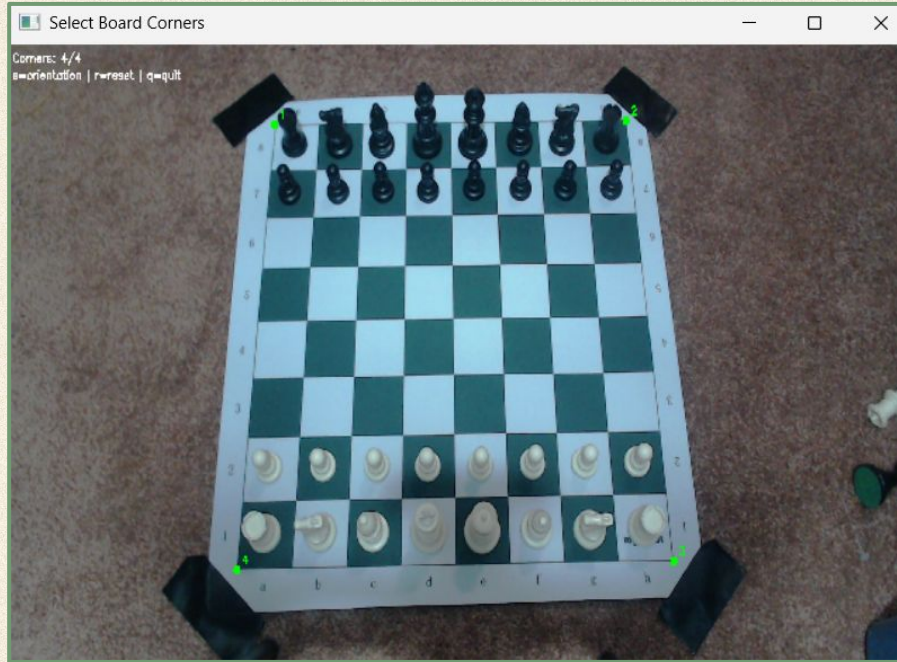




Main Changes to Vision Module

- Trained a piece classification model using a pre trained model (currently mobilenet_v3_small) and are using transfer learning via PyTorch to train it.
- We now pass in 64 images into the pieces classifier and which classifies each square and builds the FEN notation.
- Made changes to support a 4k camera
- Created an adjustable dynamic tile cropping to combat the distortion caused by the camera angle.
- Created a training image capture system and auto labeling after providing a FEN notation of the board of the current board state.
- Made changes to the manual board corner calibration to allow for rotating the rank and file of the board.

Vision Corner Calibration



Model Training (Transfer Learning)

- We currently have a labeled training data set of 2537 tile images with varied lighting conditions, and are current validation set was made using a random 80/20 split of the training images. I have plans to create a validation set with more unique positions than the training data has.
- We also train on each image 5 times with random variations in the images such as being flipped or cropped differently.
- So with the variations the total number of training images was over 10k with a validation set of over 500 of the original images.





Game Engine Pipeline

Game Engine is the brain of the system – but how does it maintain a game AND connect vision to robotics?

1. Vision FEN → validated for structure and change from prior position
2. Sliding-window filter confirms stable FEN
3. Turn change detected → legal moves simulated against board diff; special moves handled
4. Move validated → committed to BoardState
5. State machine → ROBOT_THINKING → Stockfish returns best move (UCI)
6. State machine → ROBOT_MOVING → move encoded as JSON → robot planner
7. Robot sends back signal to engine that move is done → WAITING_FOR_OPPONENT

State Machine

Engine is always in one of four states

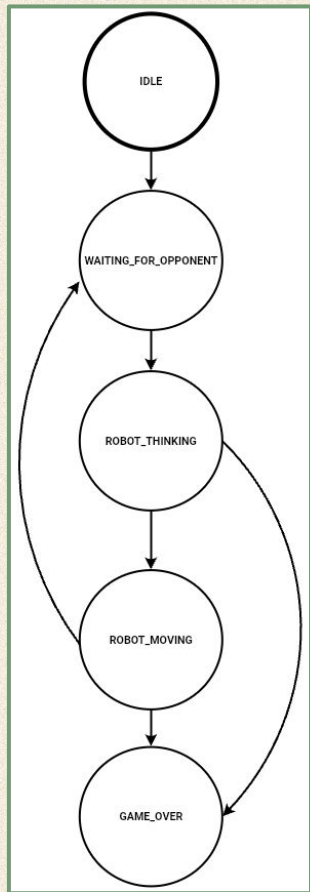
WAITING_FOR_OPPONENT → Opponent Moved → **ROBOT_THINKING**

ROBOT_THINKING → move computed → **ROBOT_MOVING**

ROBOT_MOVING → arm confirmed → **WAITING_FOR_OPPONENT**

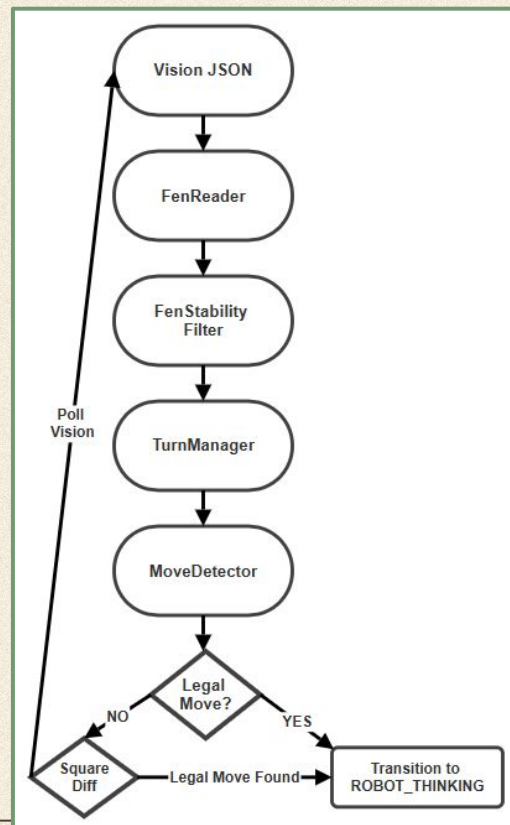
Game ends → **GAME_OVER**

- State transitions are enforced in GameStateMachine class, and invalid transitions are logged and rejected.
- Main loop ticks every 50ms, nothing executes unless the state machine allows it



FEN Stability + Validation

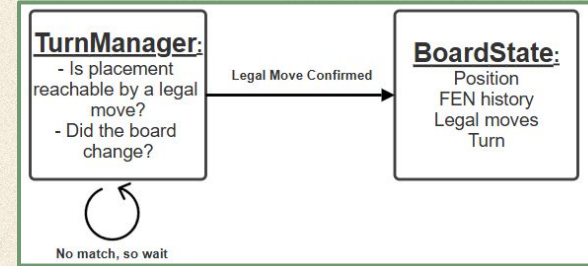
1. **FenReader** reads per-frame JSON; falls back to game_state.json if low confidence
2. **Vision outputs piece placement** — FenReader pads to full FEN
3. FenStability: **sliding window buffer**, Strict (N consecutive) or Majority (M of N) modes
4. **Stable FEN** only updates on change from last position
5. Game loop **validates FEN** — malformed readings dropped before reaching TurnManager
6. **Move detection handles piece misclassification** → accepted if any legal move matches vision placement
7. No legal match → **diff-based fallback** (squares) before rejecting



BoardState & TurnManager

Board State: Stores important board information

- Wraps python-chess library
- Tracks current position, FEN/move history, legal moves, and turn.
- All components read from BoardState object.



Turn Manager: Decision layer between raw FEN input and game logic

- Compares piece placements before and after to detect opponent move
 - If stable FEN changes, check if any legal move produces new placement
 - Legal move found → commit to BoardState and transition to ROBOT_THINKING
 - No legal move found → remain in WAITING_FOR_OPPONENT

Stockfish + Robot Handoff

Stockfish runs as child subprocess, communicate with stdin/stdout with UCI

- Skill level and time limit hardcoded for adaptive gameplay.
- If Stockfish faults, then fall back to first legal move available in position
- Move returned in UCI format → 'e2e4' → from_square, to_square

Robot Handoff

- Writes move information to robot_commands.json.
- Includes move location, moved piece, and special move type (castle, en passant) information.



```
{  
  "from_sq": "h4",  
  "to_sq": "f6",  
  "moved_piece": "q",  
  "captured_piece": null,  
  "promotion_piece": null,  
  "special": null  
}
```




Game Engine Pressure Points

- Vision module glitches can stall game
- Engine must allocate necessary time for opponent to move
- Physical issues need to be addressed
- Engine infers and reconstructs move based on two differing vision outputs
- Design more fallback layers
- More testing needs to be conducted

Robotics: Pipeline

- **Two systems:**
 - Motion Planner
 - Visualization/Controller
- Splitting the logic up allows for easier migration to the real robot after testing in the simulator is complete.
 - Motion planner and visualization are independent, planner sends joint position targets on state change.
 - Visualization simulation and real life controller both take the data in the same way, reducing the refactoring necessary when moving to the real robot.



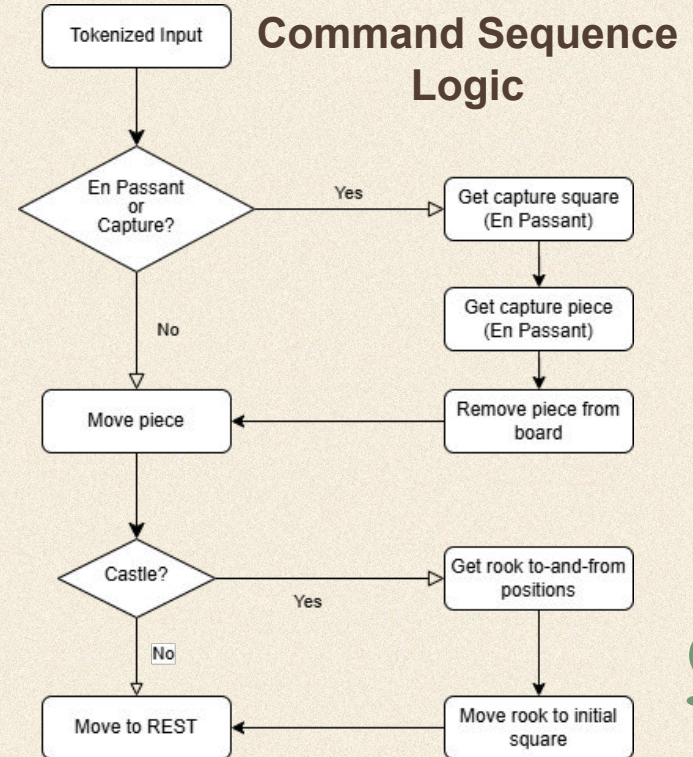
Robotics – Motion Planner

- **Board Configuration:** Configures board distance from arm base and board dimensions to construct rank and file coordinates
- **Coordinate Transformation:** Mapping algebraic notation (e2e4) to Cartesian workspace coordinates (x, y, z)
- **Path Planning:** Collision-free z-axis clearance for piece transit
- **Input Parsing:** Move string is tokenized into
 - From square
 - To square
 - Captured piece
 - Promoted piece
 - Special action
- Information gets passed to command constructor

Robotics – Motion Planner

Movement Sequence:

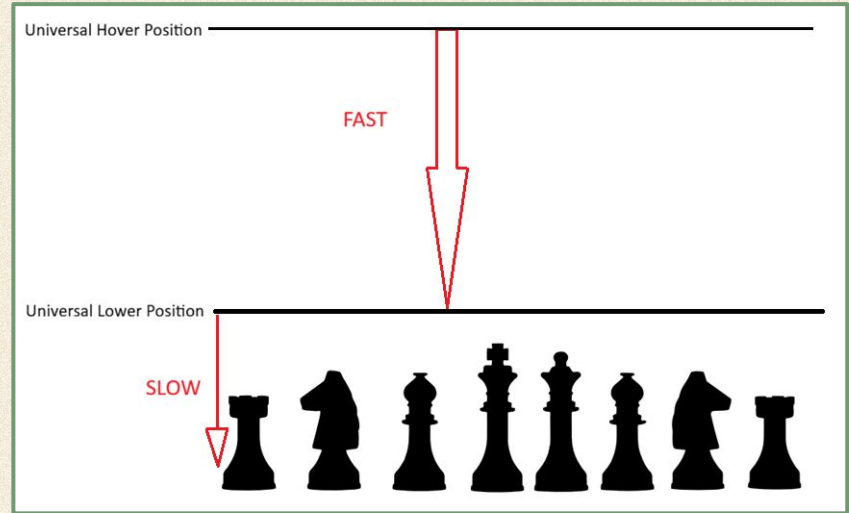
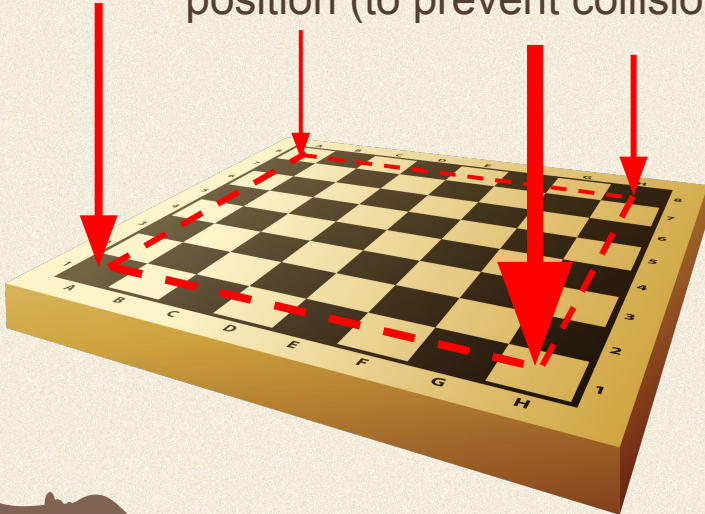
- Starts at rest position →
- Moves to high position over initial square →
- Lowers to piece (depending on piece height) →
- Grab piece →
- Lifts to high position →
- Moves to high position over target square →
- Lowers to place piece →
- Release piece →
- Lifts to high position →
- Back to resting position



Motion Planner Changes for Robot

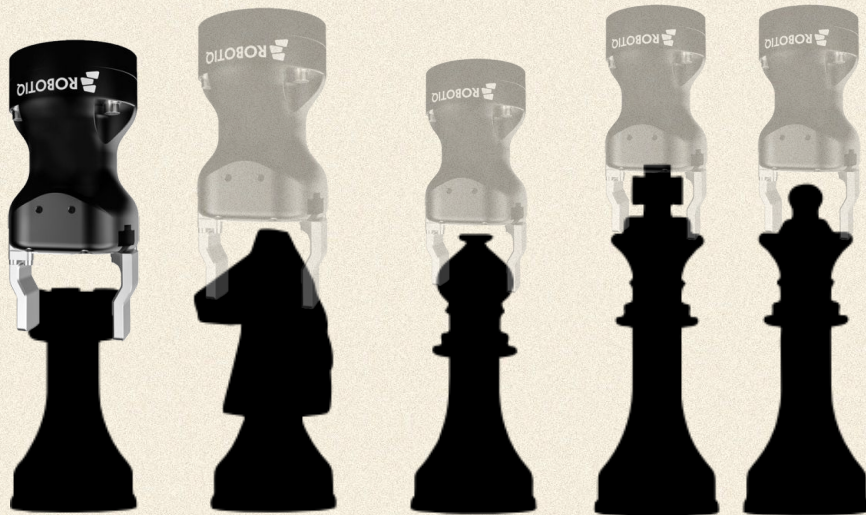
- **Board Calibration:**

- **Square Positions:** 4-Point Calibration → Forward Kinematics → Linear interpolation between 4 corner squares
- **Hover & Lower:** Calibrate hover height and universally safe lower position (to prevent collision)



Motion Planner Changes for Robot (Cont.)

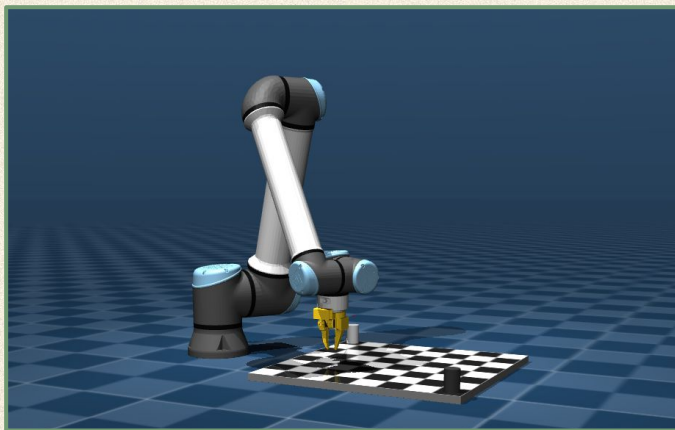
- **Gripper Calibration:**
 - **Optimal Grip:** Determine grip force and where to grip a given piece



Robotics: Visualization vs Controller

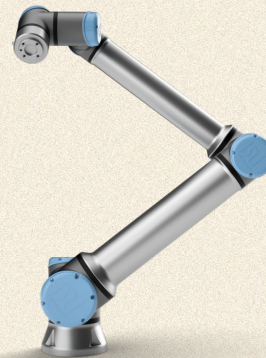
- **Mujoco Simulation**

- Fixed time step PID Control
- GUI with arm model and collidable pieces
- Plain Python, read/write to target joint position array



- **UR10e Controller**

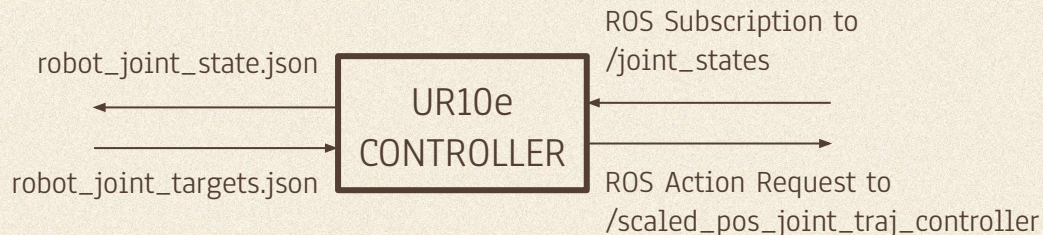
- Continuous control
- Input target motor positions, robot keeps position itself
- ROS 1, send motor position topics to the robot (Publisher), read joint positions (Subscriber)



Robotics – UR10e Controller

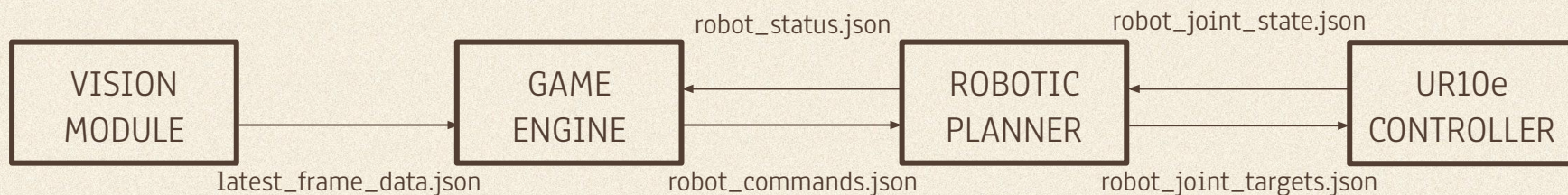
- **ROS Node**

- **Write target positions:**
 - Receives commands from planner from `robot\_joint\_target.json` as JSON object `name: target position` for each joint of the manipulator
 - Converts to the ROS topic's expected order and begins a ROS action, queuing the target positions as waypoints, completing them sequentially
- **Read current positions:**
 - Can also request the current position of the robot joints via a ROS Subscriber
 - Outputs current robot arm position to `robot\_joint\_state.json` upon request



Pipeline Orchestrator

- All subsystems set up their own communication channels, so a overarching program isn't required for the system to function.
- For convenience, a orchestrator script can run all systems at once, ensuring the launch configurations are set up properly to be compatible with each other.
- All systems have a `--auto` flag to signify listening to communication channels, but each system can still be tested independently with manual input by omitting `--auto`



Next Steps

Get into the lab → run system with UR10e arm

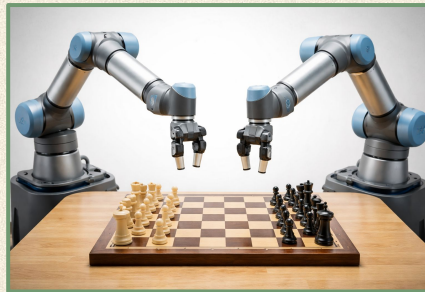
- Configure ROS and Docker setup
- Determine different angle amounts for all necessary movements
 - Picking and placing different pieces, hover, castling king/queenside
 - Gripper force and approach angles
- Chess full board and individual square calibration mechanism.
- Connect full system pipeline to ros and robot arm
- Continue training vision module
- Testing in lab hours

ROS



April Roadmap

Week	Tasks
Week 1	Initial Lab Contact, ROS Setup, Requirements Gathering
Week 2	Test chess board calibration
Week 3	Solve puzzles with whole system
Week 4	Have full-game implementation → chess tournament



The image features five chess pieces arranged around the text. At the top center is a dark brown King piece with a cross on its crown. To its left is a dark brown Rook piece. To its right is a dark brown Rook piece. Below the King is a green Bishop piece. To the right of the Bishop is a green Pawn piece. The pieces are set against a light beige background with a dark brown border.

Questions ?